

Code Analysis Report

Linear Search & Binary Search — Phone Book Program (C++)

Name: Diya Ravindra Bhansali

PRN: 125B1F271

Class & Div: SY BTech IT – A4

1. Program Overview

The submitted program (1784541051076_main.cpp) implements a simple console-based phone-book manager in C++. It stores up to 20 contacts in a global fixed-size array of a phonebook struct (a char name[20] and a std::string p_num), and offers a menu to add records, display them, and search for a contact by name using two algorithms: a Linear Search and a Binary Search. Both search routines count and report the number of comparisons performed, which is used to illustrate their time-complexity behaviour.

2. Functional Walkthrough

2.1 Data Entry (Option 1)

Repeatedly reads a name and phone number into record[ct] and increments ct until the user answers 'N' to "add more".

2.2 Display (Option 2)

Iterates $i = 0 \dots ct-1$ and prints each stored name/phone pair — an $O(n)$ traversal.

2.3 Linear Search (Option 3) — LinearSearch_name()

Scans every record from index 0 to $ct-1$ and compares it against the search key using strcmp, incrementing a comparisons counter on every iteration regardless of whether a match is found early. It continues scanning even after a match ($flag = 1$) instead of breaking, so duplicate names are all reported but the comparison count no longer reflects a true 'stop at first match' linear search.

2.4 Binary Search (Option 4) — BinarySearch_name()

Implements the classic iterative binary search with low/high/mid pointers and strcmp for ordering, breaking out as soon as a match is found. This is correctly coded as an algorithm, but see Section 3 for the critical precondition it silently violates.

3. Time & Space Complexity Analysis

Let n = number of stored contacts (ct).

3.1 Linear Search

- Best case: $O(1)$ — the target is the first record.
- Worst / average case: $O(n)$ — the target is last, absent, or (as coded) the loop always runs to completion since it never breaks on a match.
- Space: $O(1)$ — only sname and a counter are used; no extra memory scales with n .

3.2 Binary Search

- Best case: $O(1)$ — the target is the middle element.
- Worst / average case: $O(\log n)$ — the search space is halved on every iteration.

- Space: $O(1)$ for this iterative implementation (only low, high, mid, comparisons are kept); a recursive rewrite would cost $O(\log n)$ stack space instead.

3.3 Summary Table

Algorithm	Best Case	Worst / Average Case	Space Complexity
Linear Search	$O(1)$	$O(n)$	$O(1)$
Binary Search	$O(1)$	$O(\log n)$	$O(1)$ iterative, $O(\log n)$ recursive

Binary search's $O(\log n)$ bound only holds because the algorithm assumes the underlying array is sorted by the comparison key (name). That assumption is the most important correctness issue in this program — see Section 4, Issue 1.

4. Issues Found in the Implementation

#	Issue	Location	Impact
1	Binary search assumes records are sorted by name, but records are stored in insertion order with no sort step.	BinarySearch_name()	Binary search silently gives wrong / missed results whenever names are entered out of alphabetical order.
2	Fixed-size global array record[20] with no bounds check on ct.	struct phonebook record[20]; case 1	Adding a 21st contact causes a buffer overflow / undefined behavior.
3	Name stored as char name[20] and read with cin >> without a width limit.	struct phonebook; cin >> record[ct].name	A name of 20+ characters overflows the buffer (classic gets/scanf-style risk).
4	Search is case-sensitive and requires an exact match (strcmp).	LinearSearch_name(), BinarySearch_name()	"diya" will not match "Diya"; reduces usability.
5	Global mutable state (record[], ct) instead of parameters / class encapsulation.	File scope	Harder to test, reuse, or extend; all functions are tightly coupled.
6	No validation on menu input (cin >> ch); non-numeric input can put cin into a fail state / infinite loop.	main()	Program can loop printing "Invalid choice" forever or misbehave on bad input.
7	std::string is already used for phone number but char[] is used for name in the same struct.	struct phonebook	Inconsistent style; loses the safety std::string already provides for p_num.

5. Recommended Optimizations

5.1 Correctness — sort before binary search (highest priority)

Binary search is only valid on sorted data. The program never sorts record[] by name, so it happens to work in the sample run purely because the demo data (Abhay, Bina, Chetan, Diya, Hiten) was entered alphabetically. Two fixes are possible:

- Keep insertion order for display, but sort a copy (or an index array) by name immediately before running BinarySearch_name(), e.g. `std::sort(record, record+ct, cmp)` with a comparator using `strcmp`.
- Or maintain the array in sorted order at insertion time (insertion sort on add), which keeps every subsequent binary search valid at $O(\log n)$ with an $O(n)$ insertion cost.

5.2 Replace fixed array with std::vector

Replacing `phonebook record[20]` with `std::vector<phonebook>` removes the hard 20-contact ceiling and the associated overflow risk, while keeping $O(1)$ amortized append and $O(1)$ random access needed by binary search.

5.3 Use `std::string` for the name field

Switching `char name[20]` to `std::string name` removes the fixed-length buffer-overflow risk, lets comparisons use the natural `==` / `<` operators instead of `strcmp`, and keeps the struct consistent with `p_num`, which already uses `std::string`.

5.4 Stop the linear search on first match

Add a `break` after the match is found (or track and report only the comparisons needed to reach that point) so the reported comparison count reflects a genuine 'early-exit' linear search rather than always scanning the full array.

5.5 Case-insensitive comparison

Normalize both the stored names and the search key (e.g., to lowercase) before comparing, so searches like "diya" and "Diya" both succeed.

5.6 Input validation

Guard `cin >> ch` (and other numeric reads) with `cin.fail()` handling — clear the stream and ignore the bad input — to prevent an infinite "Invalid choice" loop on non-numeric input.

5.7 Reduce global state / improve modularity

Pass `record` and `ct` as function parameters (or wrap the phone book in a small class with the array/vector as a private member) instead of relying on file-scope globals. This makes the search functions independently testable and reusable.

5.8 Optional: $O(1)$ average-case lookup

If exact-name lookup is the dominant use case (rather than range queries or alphabetical listing), an `std::unordered_map<std::string, phonebook>` keyed by name would give $O(1)$ average-case search, trading away the ordering that binary search exploits. This is a design choice, not a bug fix — binary search remains the right pedagogical choice for this assignment since it demonstrates $O(\log n)$ behaviour.

6. Conclusion

The program correctly implements the mechanics of both linear and binary search and correctly reports their theoretical complexities ($O(n)$ and $O(\log n)$) in its embedded comments. Its main weakness is a correctness gap rather than a complexity one: binary search's $O(\log n)$ guarantee depends on sorted input that the program never enforces. Combined with the fixed-size buffer risks, these are the two changes (sort-before-search, and vector/string instead of raw arrays) that would most improve the program's correctness and safety without changing its educational intent.